

Handling input in Python and Nested Loops

CS 101 Algorithmic Problem Solving

Fall 2024

Up until this point, you haven't had to deal with taking input in your pseudocode. You would just refer to variables by their names and use them freely assuming their values and data type was correct. However, as you will find in your labs, that is not how things work in case of actual programming.

How to take input?

Following is the syntax for taking input in Python:

```
variable = input('Enter prompt here')
```

Quite simple, right? Whenever the interpreter comes across a line that is asking for input, the console will print the prompt (the **string** within parentheses) and the user can enter the desired input which is stored in the variable as a **string** (keep that in mind!). You also can leave the prompt empty, which is usually going to be the case when you are taking input in labs as you know what to input so the prompt doesn't matter much.

Let's take a look at an example. Suppose you want to take input from the user and output its square:

```
number = input('Enter your number: ')
number = int(number) # converting the STRING input to INTEGER to square it

print(number**2)
```

You can try this out on your own to see what the user experience will be like. Just copy the code block into VSCode.

The most important thing to keep in mind is that the input always comes in a string, unless you explicitly convert it into a different datatype. You can use typecasting functions such as `int()` and `float()` to convert it into whatever kind you want assuming it is a valid conversion (for example, "hello" cannot be converted into an `int` or `float`).

Multiple variables in one line

Throughout your worksheets, you've seen input been given as multiple separate variables all in one line; the format along the words of "a single line containing 3 space separated integers" (note that although we use the word integer, the input is still a string). You must be wondering how to separate the input, that is coming in one line, into separate variables to work with them properly. This might be disappointing to some but you won't learn how to properly do that until

you get familiar with the list datatype and list methods. Because of this, your lab input will now contain contents of separate variables in separate lines.

However, there does exist a very impractical way of separating the one line input into separate variables that utilizes just your current knowledge of programming. If you want, you can stop reading here as this method will never be asked of you and is only included in this reading for the overly curious or for those that just want to apply their skills a bit further.

Suppose you have an input coming in a single line containing three space separated integers denoting the values of variables X, Y, and Z respectively. You can do the following:

```
X = ''
Y = ''
Z = '' # create empty strings for each of the variables
XYZ = input() # take input

space1 = False
space2 = False # make flags for encountering each of the spaces

for i in XYZ:

    if i != ' ': # if i is not an empty space

        if space1 == False: # the first space hasn't been encountered
            X += i # concatenate the digit i to the X variable
        elif space1 == True and space2 == False: # only the first space has been encountered
            Y += i # concatenate the digit i to the Y variable
        elif space2 == True: # the second space has been encountered
            Z += i # concatenate the digit i to the Z variable

    elif i == ' ': # if you encounter a space

        # check to see which space it is and turn that flag to True
        if space1 == False:
            space1 = True
            # convert the X string to integer as all its digits have been concatenated
            X = int(X)
        elif space2 == False:
            space2 = True
            # convert the Y string to integer as all its digits have been concatenated
            Y = int(Y)

Z = int(Z) # convert the last string to integer as all its digits would have been concatenated
if the loop ended
```

Python Lecture: Nested Loops

(Examples: mult, stars, primetest, diamond, checkerboard)

Loops Inside of Loops

I. Analogy - Nested Loops and Tables, Multiplication Table

In most of the loop situations we have encountered so far, a loop index marches linearly in some direction, eventually stopping when it gets too big or too small. If we were to visualize each unique value of the loop index, we most likely would visualize each of the values in the order the variable takes them, in one long line.

For example, the loop structure

```
for i in range(10, 40, 3):
```

we might visualize the different values `i` equals in sequence as follows:

10 13 16 19 22 25 28 31 34 37

But, imagine that we wanted to visualize a table of some sort, such as this one:

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25

You might recognize this as a multiplication chart. Each row of the chart shows a sequence of values. Naturally, we might think of designing a loop just to print one row.

But then, we'd have to write five loops in a row just to print out this chart. One might initially come up with this to get the chart on the previous page:

```
for j in range(1, 6):
    print(j,end="\t")
print()
for j in range(1, 6):
    print(2*j,end="\t")
print()
for j in range(1, 6):
    print(3*j,end="\t")
print()
for j in range(1, 6):
    print(4*j,end="\t")
print()
for j in range(1, 6):
    print(5*j,end="\t")
print()
```

But, this seems to be a bit wasteful. What if we want to print a multiplication chart for all positive integers upto 20, would we really want to write out 20 loops, one after the other?

What we notice here is that the print itself in each of the different loops is pretty similar. In fact, the only thing that changes between each different print between successive for loops is what we multiply j by. The values we multiply j by follows a relatively straight-forward pattern: 1, 2, 3, 4, 5. If we think about our inner loop, we see that part of the point of loops is to encapsulate patterns so we don't have to physically type out so many lines of code. Thus, we can simply create a loop with a different variable to go through the integers 1, 2, 3, 4 and 5. Our resulting code is as follows:

```
for i in range(1, 6):
    for j in range(1, 6):
        print(i*j,end="\t")
    print()
```

We've found the underlying pattern in the repeated code and abstracted it away, expressing the repeated for loops by putting our initial for loop that we wrote multiple times as a single line of code inside of another for loop! Needless to say, this shortens our code and makes it much more flexible to modify. (We can very easily change this version to print out a 20 x 20 multiplication chart. The same edit on the old version would be quite tedious!)

References

Python Lecture: Nested Loops

<https://www.cs.ucf.edu/~dmarino/ucf/bhcsi/lectures/intro/Notes-NestedLoops.pdf>